



FACULTY OF COMPUTING

SESSION 2023/2024

SEMESTER 2

SECJ1023-08 PROGRAMING TECHNIQUE II

SECTION 08

Group Project Deliverable 4

Finale Deliverable

NAME:

LEE LAI QUEEN (A23CS0100)

WONG SHI YUN (A23CS0198)

CHEN SHU YAN (A23CS0059)

LECTURER NAME:

DR LIZAWATI BINTI MI YUSUF

Table of Content

1. Project Description
 - 1.1. Objectives
 - 1.2. Purpose of System
2. Analysis and Design
 - 2.1. Flow Chart
 - 2.2. UML Class Diagram
3. Classes Relationships
 - 3.1. Encapsulation
 - 3.2. Composition
 - 3.3. Aggregation
 - 3.4. Inheritance
 - 3.5. Polymorphism
 - 3.6. Array of Objects
4. Codes

1.0 Project Description

1.1 Objectives

The suggested reference book exchange system is to offer a comprehensive platform for users to share, borrow, and lend reference books to transform access to academic resources. The system's main goals and aims are listed below:

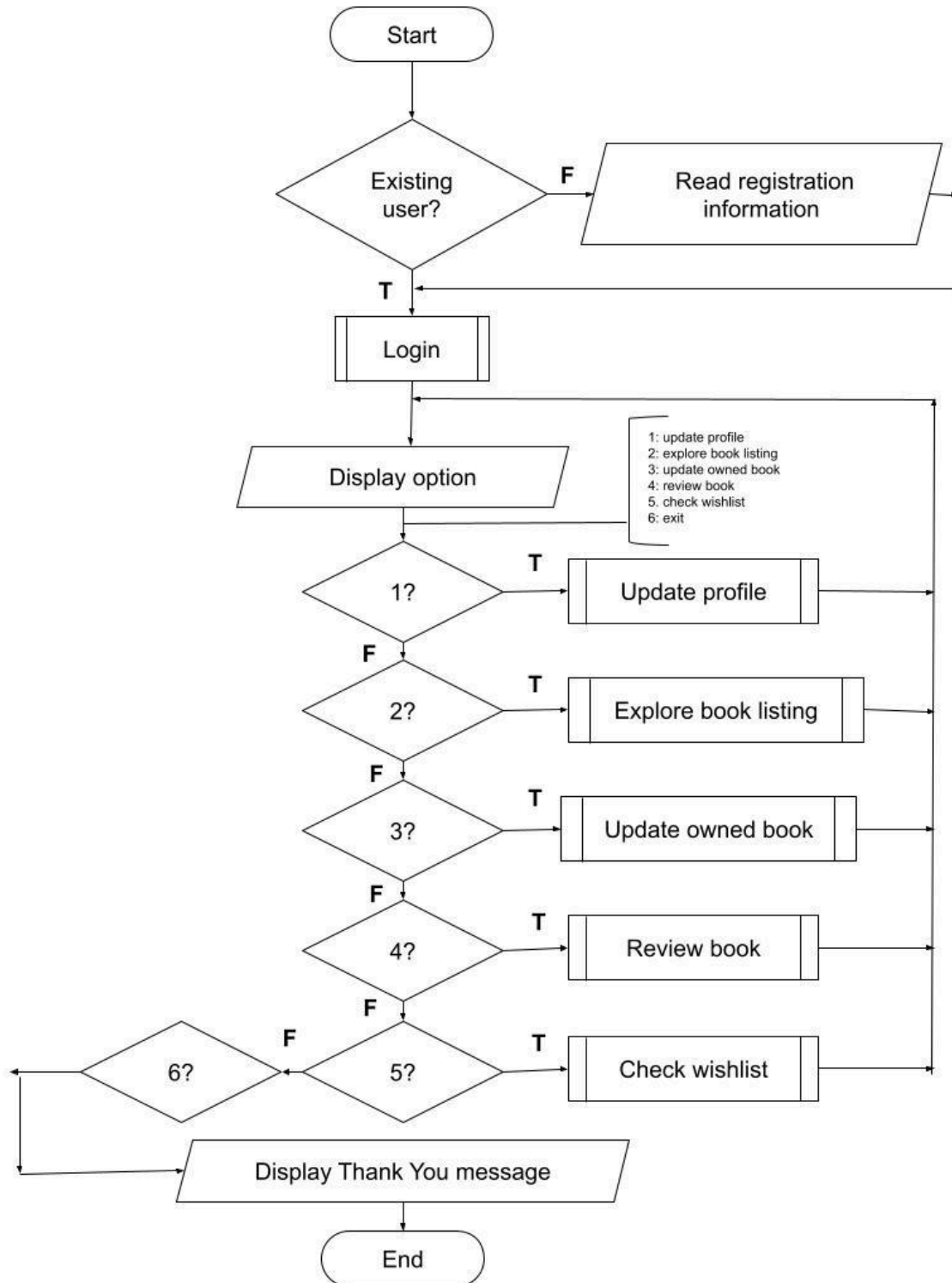
- Knowledge sharing
 - encourage the sharing of knowledge and information among society by enabling users to exchange their reference books that are no longer required
- Secure platform
 - provide a trusted platform for users interested in sharing and acquiring reference materials. To be able to share their books in a safety way
- Environmental sustainability
 - Promote sustainable practices by facilitating the reuse of existing resources and circulation of reference books. (environmental friendly)
- Facilitate learning
 - enable users with limited financial resources to gain access to more educational resources by exchanging available reference materials with other users
- Expanding the scope of sharing
 - goes beyond traditional methods of sharing books in a small community Share our books across states and throughout the whole country

1.2 Purpose of System

The purpose of this system is to help users, especially students and scholars have the access to educational resources or some limited edition reference books without spending any.

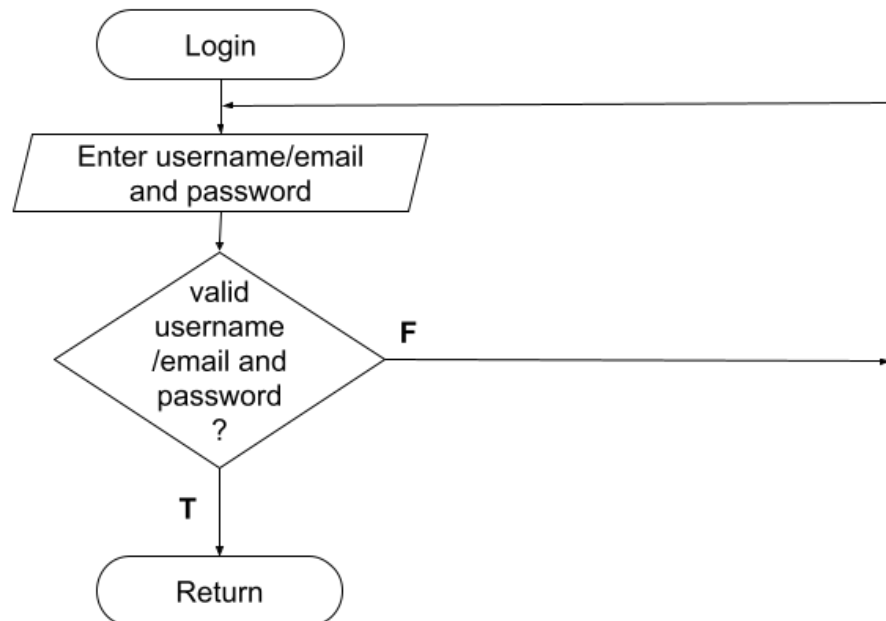
2.0 Analysis and Design

2.1 Flow Chart

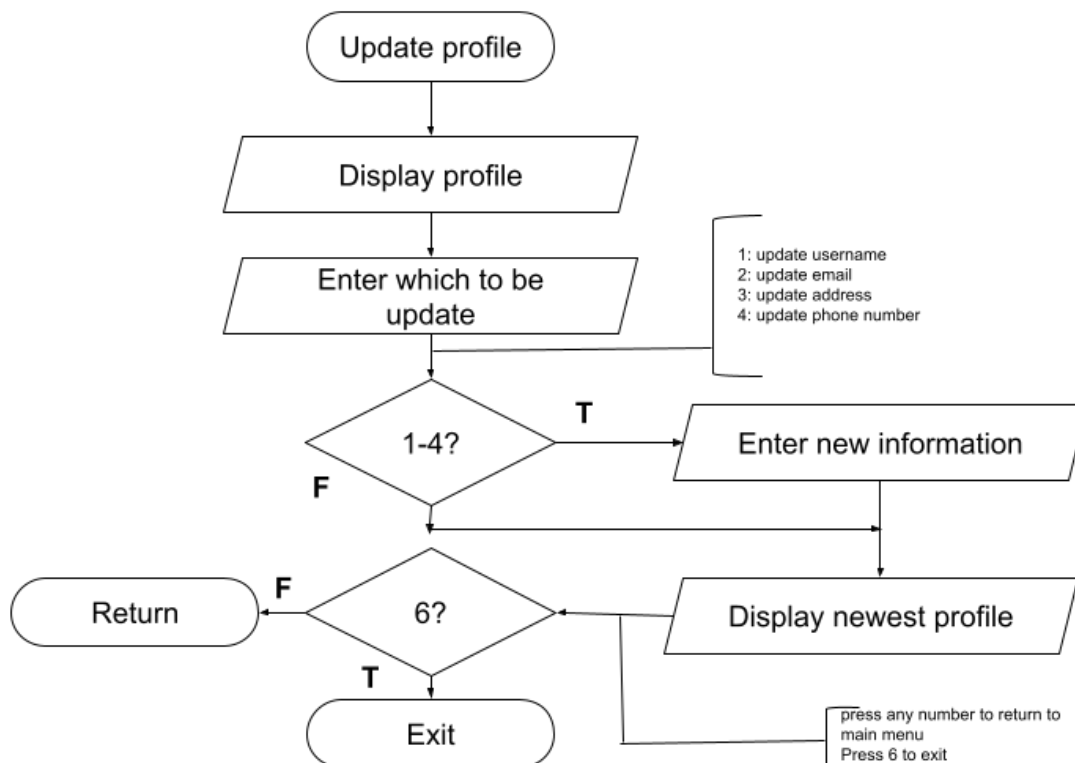


2.1 Flow Chart

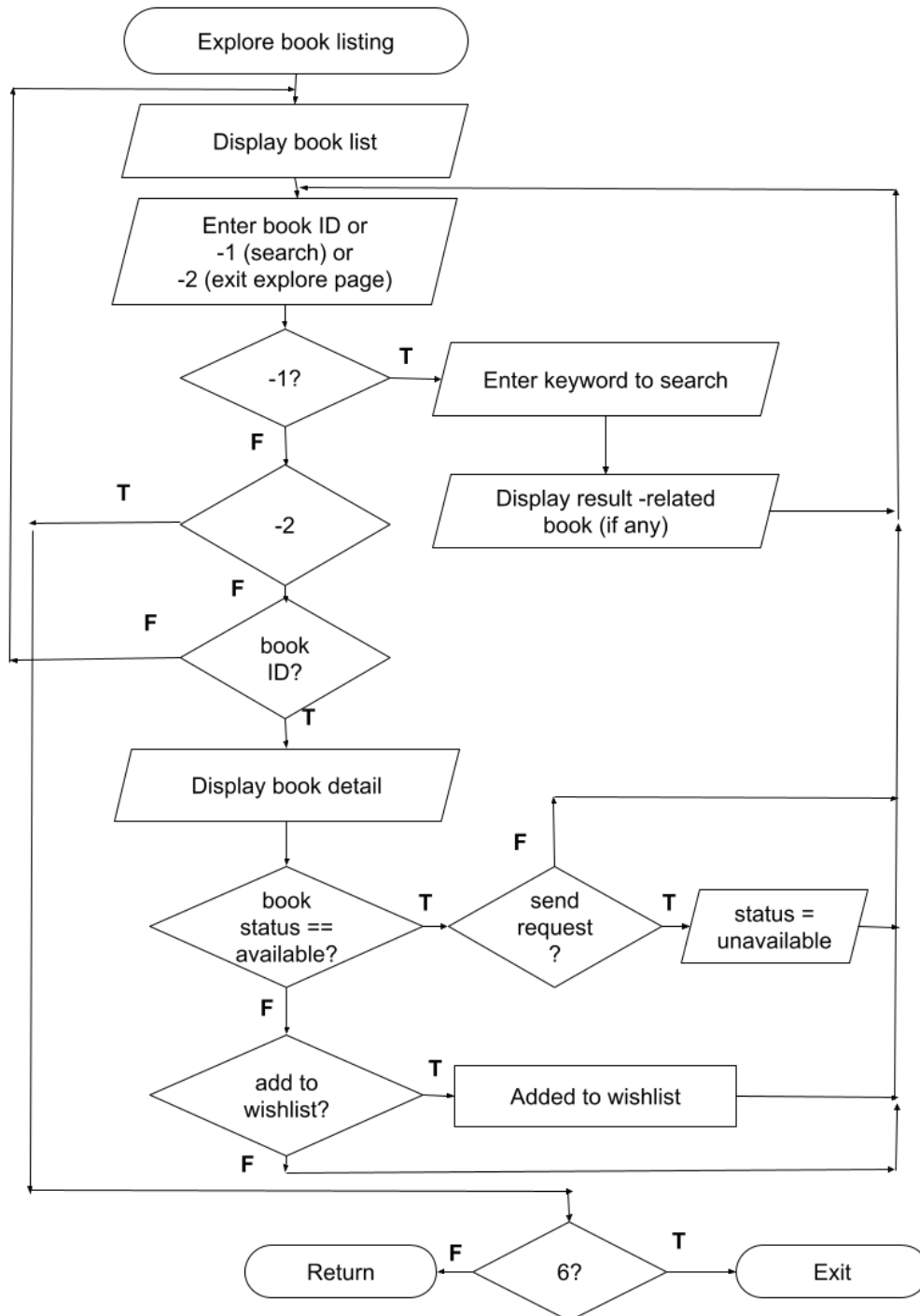
2.1.0 login



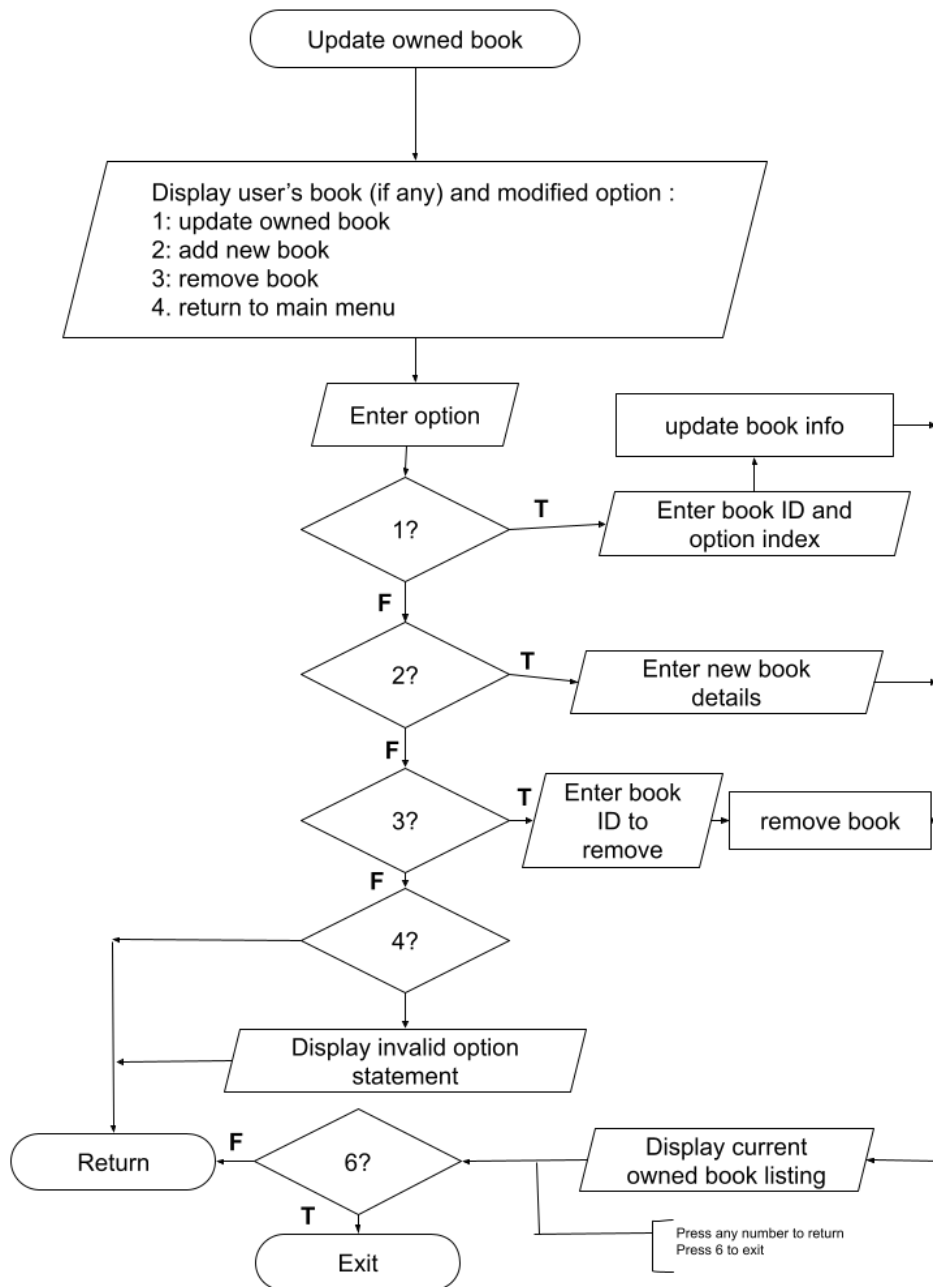
2.1.1 update profile



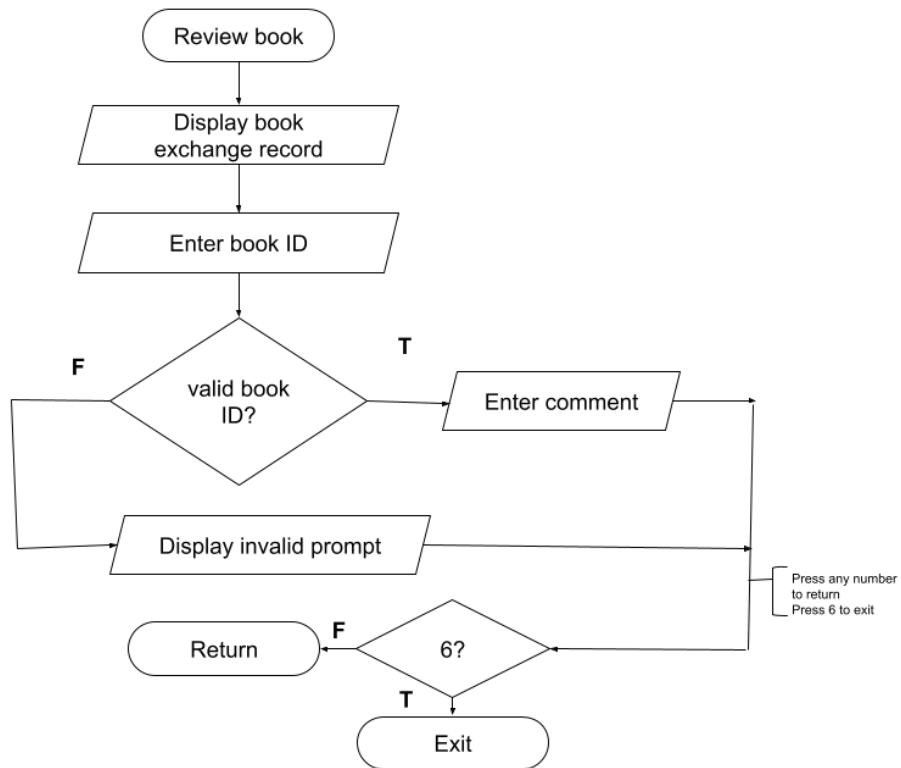
2.1.2 explore book listing



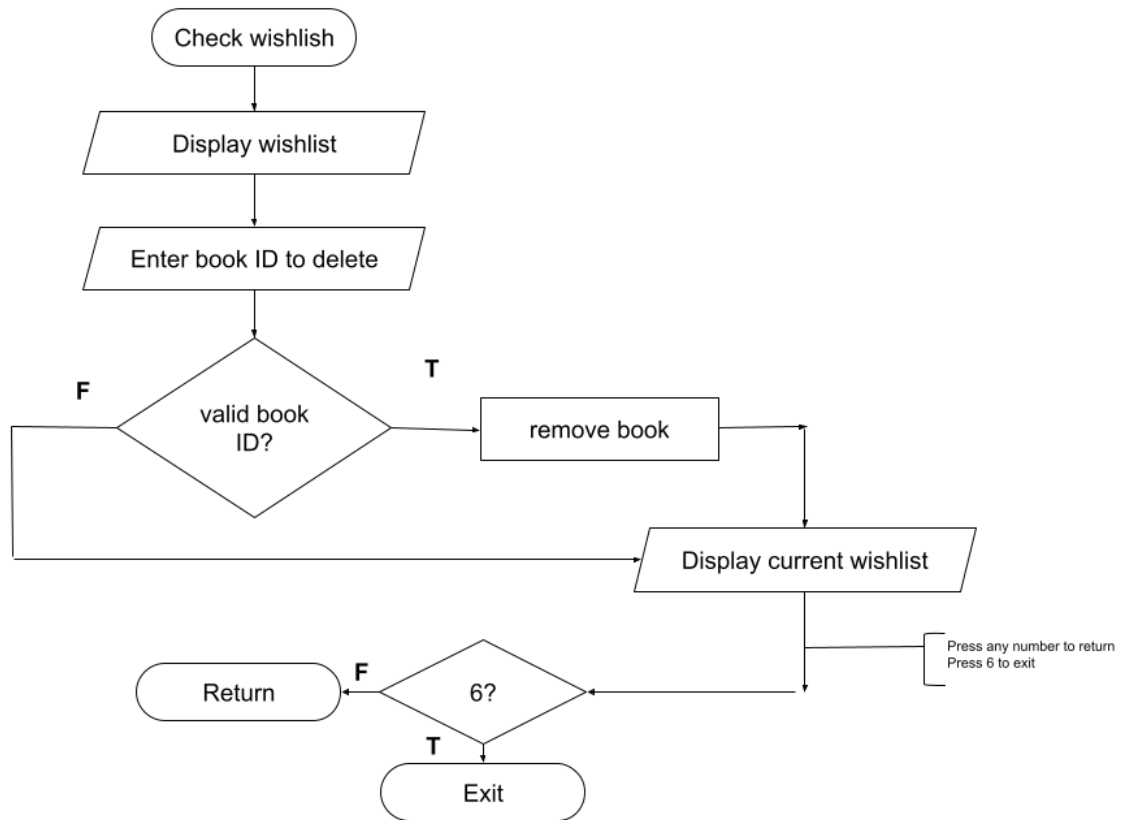
2.1.3 Update owned book listing



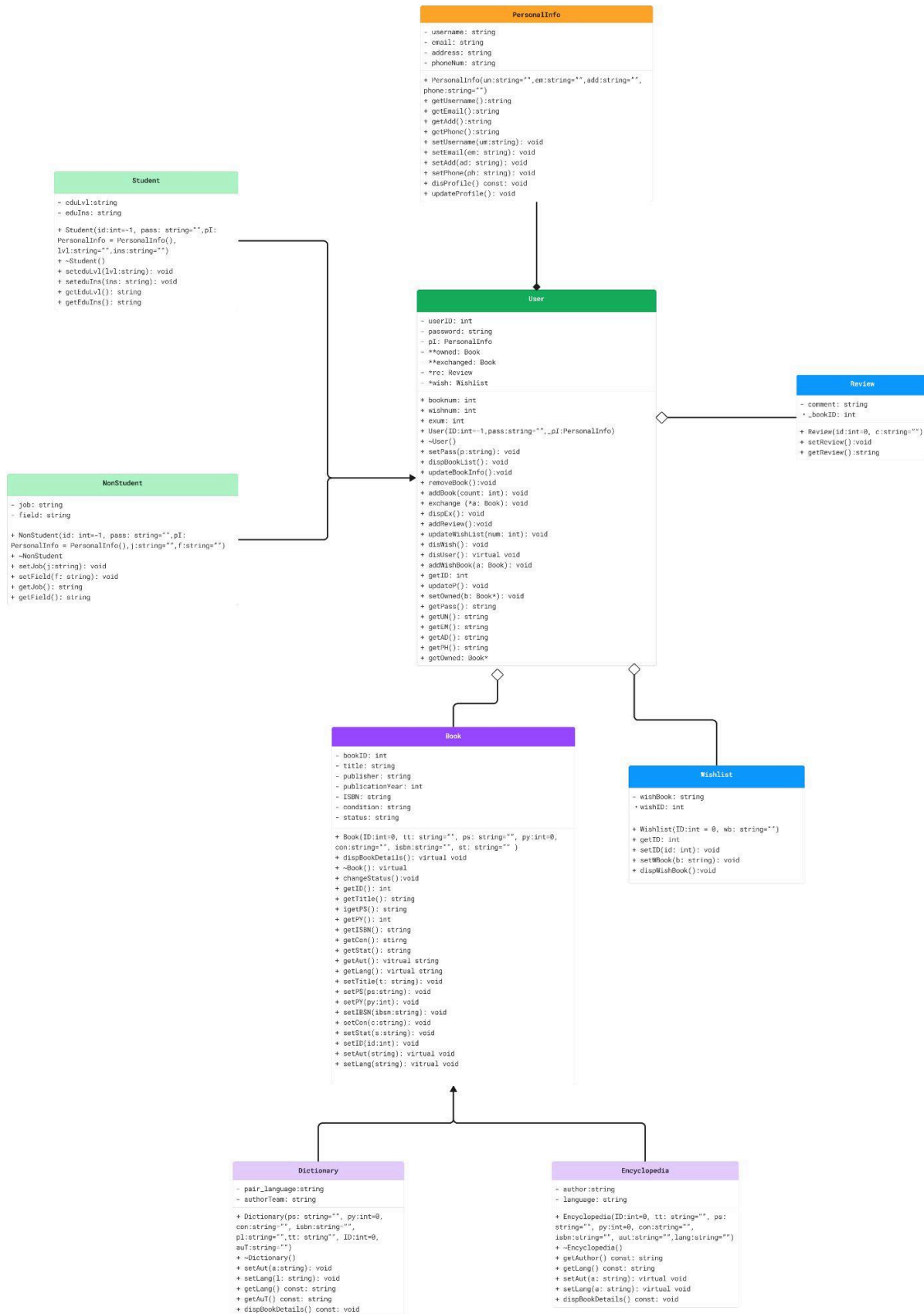
2.1.4 Review Book



2.1.5 Update wishlist



2.2 UML Class Diagram



3.0 Classes Relationships

3.1 Encapsulation

- Encapsulation is used in every class combining attributes of a class and methods of that particular class. For example, class User used encapsulation to combine all its attributes and methods. The same applies to every other class.

3.2 Composition

- User object has a personalInfo object. The personalInfo object in user object stores user's personal information such as username, email, address and phoneNum. If the user object is destroyed, its personal information should also be cleared from the system. Thus it is a composition relationship.

```
265 class User {
266     protected:
267         int userID;
268         string password;
269         PersonalInfo pI;
270         Book **owned;
271         Book **exchanged;
272         Review *re;
273         Wishlist *wish;
274 }
```

3.3 Aggregation

- User object has a book array object that stores owned books. It is a one way aggregation relationship and objects involved here are independent. The books that are exchanged to other users still exist in the exchanged user's account even if the original owner deleted its account and cleared his data.
- User object has an exchanged book array that stores information about books that the user has exchanged with other users.
- User object also has a review object that stores user's reviews. The relationship between review object and user object is aggregation because even if the user object is destroyed, the review that this user left

for any book would still be useful for any other users wishing to exchange the books reviewed in the future.

- User object has a wishlist array object that stores the title of all the books marked by the user for future reference. It is set as an aggregation relationship because more book titles will be needed to be added into it in the future, and the wishlist array object will undergo as many times of modification as the users like.

```
265 class User {
266     protected:
267         int userID;
268         string password;
269         PersonalInfo pI;
270         Book **owned;
271 +       Book **exchanged;
272         Review *re;
273         Wishlist *wish;
274
```

3.4 Inheritance

- Dictionary class and Encyclopedia class are inherited from Book class as both are types of books and share certain similar attributes, thus will inherit attributes of title, publisher, publicationYear, ISBN, condition, and status.
- Student class and NonStudent class are both inherited from User class as both share similar qualities and share similar functions, Thus both Student and NonStudent classes inherit all the attributes and methods of User class such as userID, password and others.

```
13 class Book {
14     protected:
15         int bookID;
16         string title;
17         string publisher;
18         int publicationYear;
19         string ISBN;
20         string condition;
21         string status;
```

```

98 class Dictionary: public Book {
99     private:
100         string pair_language;
101         string authorTeam;
102
103     public:
104         Dictionary(int ID=0, string tt="", string ps="", int py=0, string con="", string isbn="")
105         {
106             pair_language = pl;
107             authorTeam = auT;
108         }
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125 class Encyclopedia: public Book {
126     private:
127         string author;
128         string language;
129
130     public:
131         Encyclopedia(int ID=0, string tt="", string ps="", int py=0, string con="", string isbn="")
132         {
133             author = aut;
134             language = lang;
135         }
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163 class User {
164     protected:
165         int userID;
166         string password;
167         PersonalInfo pI;
168         Book **owned;
169         Book **exchanged;
170         Review *re;
171         Wishlist *wish;
172
173     public:
174         int booknum;
175         int wishnum;
176         int exnum;
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598 class Student: public User {
599     private:
600         string edulvl;
601         string eduIns;
602
603     public:
604         Student(int id=-1, string pass="", PersonalInfo pI=PersonalInfo(), string lvl="", string ins="")
605         {
606             edulvl = lvl;
607             eduIns = ins;
608         }
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623 class NonStudent: public User {
624     private:
625         string job;
626         string field;
627
628     public:
629         NonStudent(int id=0, string pass="", PersonalInfo pI=PersonalInfo(), string j="", string f=""): User(id, pa
630         {
631             job = j;
632             field = f;
633         }
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

3.5 Polymorphism

- In Book class, a method of dispBookDetails() is initialized and set as a virtual method meanwhile Dictionary class and Encyclopedia class also have methods of the same name and same parameter and same return type. The dispBookDetails() method in Dictionary class and Encyclopedia class will override the one in Book class when called to achieve similar but not entirely same actions.

```
13  class Book {
36      virtual void dispBookDetails() const {
37          cout << setw(11) << setprecision(6) << bookID
38              << setw(20) << title
39              << setw(13) << publisher
40              << setw(8) << publicationYear
41              << setw(15) << ISBN
42              << setw(15) << condition
43              << setw(15) << status;
44      }

98  class Dictionary: public Book {
113      void dispBookDetails() const override {
114          Book::dispBookDetails();
115          cout << setw(15) << authorTeam
116              << setw(15) << pair_language
117              << endl;
118      }
119

125  class Encyclopedia: public Book {
143      void dispBookDetails() const override {
144          Book::dispBookDetails();
145          cout << setw(15) << author
146              << setw(15) << language
147              << endl;
148      }
```

3.6 Array of Objects

- Array of objects is used in the main class when an allUser user array object is initialized to store all information of users in the system. The limit of users in allUser array object for now is 10.

```

677 int main() {
678
679     int ucount = 0; //total user count
680     User* allUser[10];
681     for (int i = 0; i < 10; ++i) {
682         allUser[i] = new User();
683     }
684
685     int user_i; //current user index
686     int bcount = 0; //total book count
687

```

4.0 Codes

```

//Group makabaka
//Lee Lai Queen A23CS0100
//Wong Shi Yun A23CS0198
//Chen Shu Yan A23CS0059

```

```

#include <iostream>
#include <iomanip>
#include <string>
using namespace std;

```

```

#define LIM 5

```

```

class Book {
protected:
    int bookID;
    string title;
    string publisher;
    int publicationYear;
    string ISBN;
    string condition;
    string status;

public:
    Book(int ID=0, string tt="", string ps="", int py=0, string
con="", string isbn="") {
        bookID = ID;
        title = tt;
        publisher = ps;
        publicationYear = py;
        condition = con;
        ISBN = isbn;
        status = "Available";
    }

```

```

}

virtual ~Book() {}

virtual void dispBookDetails() const {
    cout << setw(11) << setprecision(6) << bookID
        << setw(20) << title
        << setw(13) << publisher
        << setw(8) << publicationYear
        << setw(15) << ISBN
        << setw(15) << condition
        << setw(15) << status;
}

void changeStatus(){
    if (status == "Available") {
        status = "Unavailable";
    }else {
        status = "Available";
    }
}

int getID() const {return bookID;}
string getTitle() const {return title;}
string getPS() const {return publisher;}
int getPY() const {return publicationYear;}
string getISBN() const {return ISBN;}
string getCon() const {return condition;}
string getStat() const {return status;}
virtual string getAut() const {return "";}
virtual string getLang() const{return "";}

void setTitle(string t) {
    title = t;
}
void setPS(string ps) {
    publisher = ps;
}
void setPY(int py) {
    publicationYear = py;
}
void setISBN(string isbn) {
    ISBN = isbn;
}
void setCon(string c) {
    condition = c;
}

```

```

    }
    void setStat(string s) {
        if (s == "A") {
            status = "Available";
        }else{
            status == "Unavailable";
        }
    }
    void setID(int id){
        bookID = id;
    }

    virtual void setAut(string) {}
    virtual void setLang(string) {}

    friend class User;

};

class Dictionary: public Book {
private:
    string pair_language;
    string authorTeam;

public:
    Dictionary(int ID=0, string tt="", string ps="", int py=0,
string con="", string isbn="", string auT="", string pl=""): Book(ID,
tt, ps, py, con, isbn) {
        pair_language = pl;
        authorTeam = auT;
    }

    ~Dictionary() {}

    string getLang() const override {return pair_language;}
    string getAut() const override {return authorTeam;}

    void dispBookDetails() const override {
        Book::dispBookDetails();
        cout << setw(15) << authorTeam
            << setw(15) << pair_language
            << endl;
    }
    void setAut(string a){authorTeam = a;}
    void setLang(string l) {pair_language = l;}

```

```
};
```

```
class Encyclopedia: public Book {
    private:
        string author;
        string language;

    public:
        Encyclopedia(int ID=0, string tt="", string ps="", int py=0,
string con="", string isbn="", string aut="", string lang=""):
Book(ID, tt, ps, py, con, isbn) {
            author = aut;
            language = lang;
        }

        ~Encyclopedia() {}

        string getAut() const override {return author;}
        string getLang() const override {return language;}
        void setAut(string a){author = a;}
        void setLang(string l) {language = l;}

        void dispBookDetails() const override {
            Book::dispBookDetails();
            cout << setw(15) << author
                 << setw(15) << language
                 << endl;
        }
};
```

```
class PersonalInfo {
    protected:
        string username;
        string email;
        string address;
        string phoneNum;

    public:
        PersonalInfo(string un="", string em="", string add="", string
phone="") {
            username = un;
            email = em;
            address = add;
            phoneNum = phone;
        }
};
```

```

string getUsername() {return username;}
string getEmail() {return email;}
string getAdd() {return address;}
string getPhone() {return phoneNum;}

void setUsername(string un) {username = un;}
void setEmail(string em) {email = em;}
void setAdd(string ad) {address = ad;}
void setPhone(string ph) {phoneNum = ph;}

void disProfile() const {
    cout << left << setw(20) << "Username:" << username <<
endl
        << left << setw(20) << "Email: " << email << endl
        << left << setw(20) << "Address: " << address << endl
        << left << setw(20) << "Phone Num: " << phoneNum <<
endl;
}

void updateProfile() {
    int option;

    cout << ">>> UPDATE PROFILE INFO <<<" << endl;
    cout << "\nYou are only allowed to update the following
information.\n";
    cout << "1. Update username " << endl
        << "2. Update email " << endl
        << "3. Update address " << endl
        << "4. Update phone number " << endl;
    cout << "Enter desired category to update (1-4): ";
    cin >> option;

    if (option == 1){
        cout << "\nEnter new username: ";
        cin >> username;
    }else if (option ==2){
        cout << "\nEnter new email: ";
        cin >> email;
    }else if (option == 3){
        cin.ignore();
        cout << "\nEnter new address: ";
        getline(cin,address);
    }else if (option == 4){
        cout << "\nEnter new phone number: ";
        cin >> phoneNum;
    }
}

```

```

        }else{
            throw "***Invalid option number";
        }
    }

    friend class User;
};

class Review {
private:
    string comment;
    int _bookID;

public:
    Review(int id=0, string c=""): _bookID(id), comment(c) {}

    void setReview() {
        cout << "Enter book ID: ";
        cin >> _bookID;
        cout << "Write your review: ";
        getline(cin, comment);
    }

    string getReview(){
        return comment;
    }
};

class Wishlist {
private:
    int wishID;
    string wishBook;

public:
    Wishlist(int id=0, string wb=""): wishID(id), wishBook(wb) {}

    void dispWishBook() {
        cout << setw(11) << wishID
             << setw(20) << wishBook << endl;
    }

    int getID() {return wishID;}
    string getWBook() {return wishBook;}

    void setID(int id) {

```

```

        wishID = id;
    }
    void setWBook(string b) {
        wishBook = b;
    }

    friend class User;
};

class User {
protected:
    int userID;
    string password;
    PersonalInfo pI;
    Book **owned;
    Book **exchanged;
    Review *re;
    Wishlist *wish;

public:
    int booknum;
    int wishnum;
    int exnum;

    User(int ID=-1, string pass="", PersonalInfo _pI =
PersonalInfo()){
        userID = ID;
        password = pass;
        pI = _pI;
        owned = NULL;
        re = NULL;
        wish = NULL;
        booknum = 0;
        wishnum = 0;
        exnum = 0;
        owned = new Book*[LIM];

        for (int i = 0; i < LIM; ++i) {
            owned[i] = nullptr; // Initialize array elements to
nullptr
        }

        exchanged = new Book*[LIM];
        re = new Review[LIM];
        wish = new Wishlist[LIM];
    }

```

```

~User() {
    delete[] owned;
    delete[] exchanged;
    delete[] re;
    delete[] wish;
}

void setPass(string p) {
    password = p;
}

void dispBookList() {
    for (int i=0; i < booknum; i++){
        owned[i]->dispBookDetails();
    }
}

void updateBookInfo() {
    int temp;
    int select;

    cout << "Enter book ID to update book info: ";
    cin >> temp;

    for (int i=0; i<booknum; i++){
        if (temp == owned[i]->getID()){
            select = i;
            int opup;
            string New;
            int NNew;
            cout << "1. Update book title\n"
                 << "2. Update publisher\n"
                 << "3. Update publish year\n"
                 << "4. Update author\n"
                 << "5. Update language\n"
                 << "6. Update condition\n"
                 << "7. Update status\n";
            cout << "Enter book detail option to update: ";
            cin >> opup;

            if (opup == 1){
                cin.ignore();
                cout << "Enter new title: ";
            }
        }
    }
}

```

```

        getline(cin,New);
        owned[select]->setTitle(New);
        cout << "Book #" << temp << " succefully
updated.\n";

    }else if (opup == 2){
        cout << "Enter new publisher: ";
        cin.ignore();
        getline(cin,New);
        owned[select]->setPS(New);
        cout << "Book #" << temp << " succefully
updated.\n";

    }else if (opup == 3){
        cout << "Enter new publication year: ";
        cin >> NNew;
        owned[select]->setPY(NNew);
        cout << "Book #" << temp << " succefully
updated.\n";

    }else if (opup == 4){
        cin.ignore();
        cout << "Enter new author: ";
        getline(cin,New);
        owned[select]->setAut(New);
        cout << "Book #" << temp << " succefully
updated.\n";

    }else if (opup == 5){
        cin.ignore();
        cout << "Enter new language or pair language:
";

        getline(cin,New);
        owned[select]->setLang(New);
        cout << "Book #" << temp << " succefully
updated.\n";

    }else if (opup == 6){
        cin.ignore();
        cout << "Enter latest condition: ";
        getline(cin,New);
        owned[select]->setCon(New);
        cout << "Book #" << temp << " succefully
updated.\n";

```

```

        }else if (opup == 7){
            cin.ignore();
            owned[select]->changeStatus();
            cout << "Book #" << temp << "succefully
updated.\n";

        }else{
            cout << "***Invalid update option" << endl;
        }
    }else{
        cout << "\nYou don't own this book.\n";
    }
}

}

void removeBook() {
    int del;
    int s = -1;
    cout << "Enter book ID to remove: ";
    cin >> del;
    for (int i=0; i<booknum; i++){
        if (del == owned[i]->getID()){
            s = i;
            break;
        }
    }
    if (s != -1){
        for (int i=s; s<booknum; s++){
            owned[i] = owned[i+1];
        }
        booknum -= 1;
        cout << "Book #" << del << " removed from owned book
list\n";
    }else{
        cout << "You do not own this book.\n";
    }
}

void addBook(int count){

    if (booknum > LIM) {
        cout << "Book capacity reached.\n";
        return;
    }

```

```

int type;
string tempT,tempA,tempL,tempC,tempPS,tempISBN;
int tempPY;

cout << "Is your book a dictionary (1) or encyclopedia
(2)? ";

cin >> type;
cin.ignore();

if (type == 1){
    owned[booknum] = new Dictionary(count);
}else{
    owned[booknum] = new Encyclopedia(count);
}

cout << "Enter new book title: ";
getline(cin,tempT);
owned[booknum]->setTitle(tempT);

cout << "Enter new book author: ";
getline(cin,tempA);
owned[booknum]->setAut(tempA);

cout << "Enter new book language: ";
cin >> tempL;
owned[booknum]->setLang(tempL);

cout << "Enter publisher: ";
cin >> tempPS;
owned[booknum]->setPS(tempPS);

cout << "Enter publisher year: ";
cin >> tempPY;
owned[booknum]->setPY(tempPY);
cin.ignore();

cout << "Enter ISBN code: ";
cin >> tempISBN;
owned[booknum]->setISBN(tempISBN);

cout << "Enter new book condition: ";
cin.ignore();
getline(cin,tempC);
owned[booknum]->setCon(tempC);
owned[booknum]->setStat("Available");
booknum++;

```

```

        cout << "New book successfully added.\n";
    }

    void exchange(Book *a){
        exchanged[exnum] = a;
        exnum++;
    }

    void addReview() {

        if (exnum == 0) {
            string x;
            cout << "\nYou have not exchanged any book yet.\n";
            return;
        }

        int ID;
        string temp;
        cout << "\nEnter book ID of desired book to review: ";
        cin >> ID;

        for (int i=0; i<exnum; i++){
            if (ID == exchanged[i]->getID()){
                cin.ignore();
                cout << "Write your review: ";
                getline(cin,temp);
                exchanged[i]->setCon(temp);
                dispEx();
            }else{
                cout << "You are not allowed to review books that
you have not exchanged.\n";
            }
        }
    }

    void dispEx() {
        cout << "\n>>> EXCHANGE BOOK LIST <<<\n";
        cout << setw(11) << "Book ID"
            << setw(20) << "Title"
            << setw(13) << "Publisher"
            << setw(8) << "Year"
            << setw(15) << "ISBN"
            << setw(15) << "Condition"

```

```

        << setw(15) << "Status"
        << setw(15) << "Author"
        << setw(15) << "Language" << endl;
    for (int i=0;i<exnum;i++){
        exchanged[i]->dispBookDetails();
    }
}

void updateWishList(int num) {
    char op;
    int del;
    cout << "\nWould you like to delete any book from your
wishlist? (Y/N) ";
    cin >> op;

    if (op == 'Y'){
        cout << "Enter book ID to delete: ";
        cin >> del;

        for (int i=0; i<wishnum; i++){
            if (del == wish[i].getID()){
                for (int i=0; i<wishnum; i++){
                    wish[i] = wish[i+1];
                }
                wishnum --;
                break;
            }
        }
    }else{
        return;
    }
}

void disWish(){
    cout << "\n>>> CURRENT WISHLIST <<<\n";
    cout << setw(11) << "Book ID"
        << setw(20) << "Title" << endl;
    for (int i=0;i<wishnum;i++){
        wish[i].dispWishBook();
    }
}

void disUser() const {
    cout << left << setw(20) << "User ID : " << userID <<
endl;

```

```

        cout << left << setw(20) << "Password: " << password <<
endl << endl;
        pI.disProfile();
        cout << endl;
    }

```

```

void addWishBook(Book a) {
    wish[wishnum].setID(a.getID());
    wish[wishnum].setWBook(a.getTitle());
    wishnum++;
}

```

```

int getID() {return userID;}
string getPass() {return password;}
string getUN() {return pI.getUsername();}
string getEM() {return pI.getEmail();}
string getAD() {return pI.getAdd();}
string getPH() {return pI.getPhone();}
Book * getOwned() {return *owned;}

```

```

void updateP() {
    try{
        pI.updateProfile();
    }
    catch (const char* mesg){
        cout << mesg;
    }
}
void setOwned(Book* b) {
    if (booknum < LIM) {
        owned[booknum++] = b;
    } else {
        cout << "Book Limit reached\n" << endl;
    }
}

```

```

};

```

```

class Student: public User {
private:
    stringedulvl;
    stringeduIns;

public:

```

```

        Student(int id=-1, string pass="", PersonalInfo
pI=PersonalInfo(), string lvl="", string ins=""): User(id,pass,pI){
           edulvl = lvl;
            eduIns = ins;
        }

        ~Student() {}

        void seteduLvl(string lvl) {
           edulvl = lvl;
        }

        void seteduIns(string ins) {
            eduIns = ins;
        }

        string getEduLvl() {return edulvl;}
        string getEduIns() {return eduIns;}
};

```

```

class NonStudent: public User {
private:
    string job;
    string field;

public:
    NonStudent(int id=0, string pass="", PersonalInfo
pI=PersonalInfo(), string j="", string f=""): User(id, pass, pI) {
        job = j;
        field = f;
    }

    ~NonStudent() {}

    void setJob(string j) {
        job = j;
    }

    void setField(string f) {
        field = f;
    }

    string getJob() {return job;}
    string getField() {return field;}
};

```

```

void printHeader() {
    cout << "\n"
        << setw(11) << "Book ID"
        << setw(20) << "Title"
        << setw(13) << "Publisher"
        << setw(8) << "Year"
        << setw(15) << "ISBN"
        << setw(15) << "Condition"
        << setw(15) << "Status"
        << setw(15) << "Author"
        << setw(15) << "Language" << endl;
}

void exitM() {
    cout << "\nPress any number to return to main menu.\n";
    cout << "Press 6 to exit." << endl;
}

int main() {

    int ucount = 0; //total user count
    User* allUser[10];
    for (int i = 0; i < 10; ++i) {
        allUser[i] = new User();
    }

    int user_i; //current user index
    int bcount = 0; //total book count

    //initial users
    Student s1 = {ucount,"hey",{ "Ahmad","ahmad@123.com","Jalan
Raya","0126123548"},"University","UTM"};
    allUser[ucount++]=&s1;
    NonStudent n1 = {ucount,"HEY",{ "Ali","ali@123.com","Jalan
Maju","01256985"},"Teacher","Science"};
    allUser[ucount++]=&n1;

    //initial books
    Encyclopedia* b1 = new Encyclopedia{bcount++,"Science
Book","Pelangi",2024,"Brand new","2024152634","Mohamed
Ali","English"};
    Dictionary* b2 = new Dictionary{bcount++,"Chinese
Dictionary","Oxford",2012,"Old","589658654","John
Wong","Chinese-English"};

```

```

allUser[s1.getID()]->setOwned(b1);
allUser[n1.getID()]->setOwned(b2);


//login
string tempName;
string tempPass;


char existYN;
cout << "Do you have an existing account? (Y/N) ";
cin >> existYN;
cin.ignore();

if (existYN == 'N') {
    cout << "\n>>> CREATE NEW ACCOUNT <<<\n";

    string uname, email, add, phoneN, pass;
    cout << "Enter new username: ";
    cin >> uname; //one word, no space
    cout << "Enter email: ";
    cin >> email; //one word, no space
    cin.ignore();
    cout << "Enter current address: ";
    getline(cin, add);
    cout << "Enter phone number: ";
    cin >> phoneN; //one word, no space
    cout << "Set password: ";
    cin >> pass; //one word, no space
    PersonalInfo p(uname, email, add, phoneN);

    char s;
    cout << "\nAre you a student? (Y/N) ";
    cin >> s;
    cin.ignore(); // Clear newline character from buffer

    if (s == 'Y') {
        string lvl, ins;
        cout << "Enter education level: ";
        getline(cin, lvl);
        cout << "Enter education institution: ";
        getline(cin, ins);
        Student s(uaccount, pass, p, lvl, ins);
    }
}

```

```

        allUser[ucount++] = &s;
        existYN = 'Y';

    } else {
        string job, field;
        cout << "Enter job: ";
        getline(cin, job);
        cout << "Enter interested field: ";
        getline(cin, field);

        NonStudent n(ucount, pass, p, job, field);
        allUser[ucount++] = &n;
        existYN = 'Y';
    }
}

cout << endl << ">>> LOGIN <<<" << endl;
cout << "Username: ";
getline(cin, tempName);

cout << "Password: ";
getline(cin, tempPass);

int unfound=0;
for (int i=0; i<ucount; i++){
    if (tempName == allUser[i]->getUN()){
        if (tempPass == allUser[i]->getPass()) {
            cout << "Login successful! \n";
            user_i = i;
        }else{
            cout << "Wrong password\n";
            return 0;
        }
    }else{
        unfound++;
    }
}
if (unfound >= ucount){
    cout << "Wrong username\n";
    return 0;
}

```

```
//menu
```

```

int option = 0; //0 = menu

while (option != 6) {
    cout << "\n>>> MENU <<< \n";
    cout << "Option 1. Update profile\n"
        << "Option 2. Explore book listing\n"
        << "Option 3. Update owned book listing\n"
        << "Option 4. Review book\n"
        << "Option 5. Check wishlist\n"
        << "Option 6. EXIT\n";
    cout << "Press option 1-6 to proceed: ";
    cin >> option;

    //update profile
    if (option == 1) {
        cout << "\n>>> CURRENT PROFILE <<<\n";
        allUser[user_i]->disUser();

        cout << "\n";
        allUser[user_i]->updateP();

        cout << "\n\n >>> NEWEST PROFILE <<<\n";
        allUser[user_i]->disUser();

        exitM();
        cin >> option;
    }

    //explore book
    else if (option == 2) {
        int opEx;
        int seeID;
        bool contx = 1;

        while (contx){
            cout << "\n>>> EXPLORE BOOKS <<<\n";
            printHeader();
            for (int i=0; i<ucount; i++){
                allUser[i]->dispBookList();
            }

            cout << endl;
            cout << "Enter book ID to see details or press -1 to
search or press -2 to exit explore page: ";
            cin >> opEx;

```

```

        if (opEx == -2){
            contx = 0;
            break;

        }else if (opEx == -1){
            string search;
            cout << "Enter ONE search keyword for title: ";
            cin >> search;

            printHeader();

            bool foundAny = 0;
            for (int i=0; i<ucount; i++){
                for (int j=0; j<allUser[i]->booknum; j++){
                    int found =
allUser[i]->getOwned()[j].getTitle().find(search);
                    if (found != std::string::npos){

allUser[i]->getOwned()[j].dispBookDetails();
                        foundAny = 1;
                    }
                }
            }

            if (foundAny == 0) {
                seeID = -3;
                cout << "\n**No result found.\n";
            }else{
                cout << "\nEnter book ID to see details: ";
                cin >> seeID;
            }

        }else{
            seeID = opEx;
        }

        if (seeID != -3){
            for (int i=0; i<ucount; i++){
                for (int j=0; j<allUser[i]->booknum; j++){
                    if (seeID ==
allUser[i]->getOwned()[j].getID()){
                        cout << "\n>>> BOOK DETAILS <<<\n";
                        cout << left << setw(11) << "Book ID:
" << allUser[i]->getOwned()->getID() << endl;

```

```

        cout << left << setw(11) << "Title: "
<< allUser[i]->getOwned()->getTitle() << endl;
        cout << left << setw(11) << "Author: "
<< allUser[i]->getOwned()->getAut() << endl;
        cout << left << setw(11) << "Language: "
<< allUser[i]->getOwned()->getLang() << endl;
        cout << left << setw(11) <<
"Condition: " << allUser[i]->getOwned()->getCon() << endl;
        cout << left << setw(11) << "Status: "
<< allUser[i]->getOwned()->getStat() << endl;

        if
(allUser[i]->getOwned()[j].getStat() == "Available"){
            char req;
            cout << "\nWould you like to send
exchange request (Y/N)? ";

            cin >> req;

            if (req == 'Y'){

allUser[i]->getOwned()[j].changeStatus();

allUser[user_i]->exchange(&allUser[i]->getOwned()[j]);
                cout << "Exchange request
successfully sent!\n";

            }
            break;

        }else{
            char wi;
            cout << "\nWould you like to add
into wishlist (Y/N)? ";

            cin >> wi;

            if (wi == 'Y'){

allUser[user_i]->addWishBook(allUser[i]->getOwned()[j]);
                cout << "New book successfully
added to wishlist!\n";

            }

            break;

        }
    }
}

```

```

        }
    }
}
exitM();
cin >> option;
}

//update owned book
else if (option == 3){
    int opUp;
    bool conty = 1;

    while (conty){

        cout << "\n>>> CURRENT OWNED BOOK LISTING <<<\n";
        printHeader();
        allUser[user_i]->dispBookList();

        cout << "\n>>> UPDATE BOOK LISTING <<<" << endl;
        cout << "Choose among options below\n";
        cout << "1. Update owned books\n";
        cout << "2. Add new book\n";
        cout << "3. Remove book\n";
        cout << "4. Return to main menu\n";
        cout << "Enter update option: ";
        cin >> opUp;
        cout << endl;

        if (opUp == 1){
            allUser[user_i]->updateBookInfo();
        }else if (opUp == 2){
            allUser[user_i]->addBook(bcount++);
        }else if (opUp == 3){
            allUser[user_i]->removeBook();
            bcount--;
        }else if (opUp == 4){
            conty = 0;
            break;
        }
        else{
            cout << "**Invalid option choice\n";
        }
    }

    exitM();
    cin >> option;
}

```

```

    }

    //review book
    else if (option == 4){
        allUser[user_i]->dispEx();
        allUser[user_i]->addReview();
        exitM();
        cin >> option;
    }

    //display wishlist
    else if (option == 5){
        cout << "\n>>> CHECK WISHLIST <<<\n";
        allUser[user_i]->disWish();
        cout << endl;
        allUser[user_i]->updateWishList(bcount);
        cout << endl;
        allUser[user_i]->disWish();

        exitM();
        cin >> option;
    }

    //exit
    else if (option == 6){
        cout << "\nThank you for visiting.";
        return 0;
    }

    else{
        cout << "**Invalid option!";
    }

}

}

```